ELEC327 Final Project - Cubington

May 15, 2026

Zak Hashmi, Milan Addepalli, Kasim Dawood, Nathan Cao

A lab report detailing the different
components and processes for the final project

Rice University ECE

Contents

1 Introduction

Our project involves creating a joystick utilizing a Hall effect-based joystick to output direction (horizontal and vertical) signals to the MSPM0. The signals are read and interpreted by using the ADC module. These values will subsequently be mapped to a 2D direction, which is then used as an input to an Adafruit display (this particular screen communicates with the MSPM0 through SPI).

As a high-level overview, our project will consist of the following components (there are additional parts to this project, but specifically these are central to our initial goals):

Custom-designed PCB
Sparkfun Electronics Analog Switch Joystick
Adafruit TFT LCD Touchscreen

The basic premise of the game is that the screen will provide a random orientation of a cube. The user is able to control another cube displayed on the screen and the goal is to match the geometry orientation of cube number 2 with the provided cube. The goal of the game is to match as many orientations as possible within a minute.

2 Joystick

The joystick is one of the core components of the game. The user uses the horizontal and vertical axes of the joystick to orient the cube provided on the screen. Shown below is a real-life representation of the joystick.

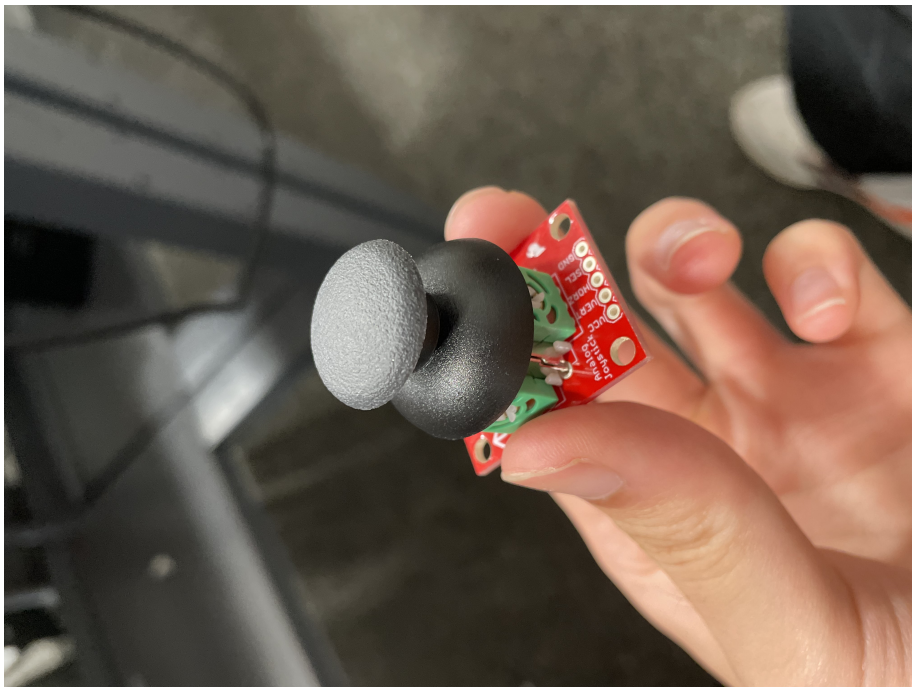

Figure 1: Physical Joystick

2.1 Physical Design Considerations

The joystick has 3 core components:

- Horizontal axis
- Vertical axis

- Button

Each axis works like a potentiometer, meaning it behaves like a variable voltage divider. The joystick is connected between **VDD** and **GND**, and as the stick moves, the wiper output for each axis changes voltage depending on its position. Those analog voltages are sent to the MSPM0 ADC pins, where the microcontroller ocnverts them into digital values that can be used in software to detect joystick direction and movement.

In order to yield stable voltage values and smooth out noise, we considered adding capacitors from the joystick signal lines to ground. However, we decided not to include them because the joystick movement does not require extremely precise analog filtering, and the small amount of noise can be handled in software.

The specific pins and their functions will be discussed more in the PCB design section of this report.

2.2 Code Implementation

2.2.1 Initialization

As mentioned above, the joystick's X and Y axes behave like potentiometers: each axis outputs a voltage between GND and VDD depending on the stick position. The code configures two MSPM0 pins as analog inputs, then uses ADC0 to repeatedly sample those voltages. Shown below is how we initialize the starting positions of the X, and Y axes:

```
volatile uint16_t joystick_raw[ADC_NUM_CHANNELS] = {0,0};
```

2.2.2 Update Joystick State

The ADC is set up in sequence mode so it automatically samples both joystick channels one after the other. After initialization, joystick_update() waits until the second conversion result is ready, then reads both ADC memory registers into the joystick_raw array. The helper functions joystick_x() and joystick_y() subtract 128 from the raw value, which recenters the joystick around zero. Notice that we use **DL_ADC12** which is Texas Instrument's library for converting 12-bit analog to digital converter.

```
void joystick_update(void) {

    while (!DL_ADC12_getRawInterruptStatus(ADC0, DL_ADC12_INTERRUPT_MEM1_RESULT_LOADED));
    joystick_raw[JOYSTICK_X] = DL_ADC12_getMemResult(ADC0, DL_ADC12_MEM_IDX_0);
    joystick_raw[JOYSTICK_Y] = DL_ADC12_getMemResult(ADC0, DL_ADC12_MEM_IDX_1);

    DL_ADC12_clearInterruptStatus(ADC0, DL_ADC12_INTERRUPT_MEM1_RESULT_LOADED);
}
```

2.2.3 Determining Direction

The direction logic portion of the code uses a "dead zone" to avoid false movement when the joystick is near the center. The code chooses to ignore values within JOYSTICK_DEADZONE, and if the joystick is outside that range, the code compares the size of the X and Y movement and chooses the dominant axis. The joystick_magnitude() function also calculates how far the joystick is from center using the distance formula.

```
Direction joystick_getDirection(void) {
    int16_t x = joystick_x();
    int16_t y = joystick_y();
```

```
4
5       // Return none if we're within the deadzone
6       if (x > -JOYSTICK_DEADZONE && x < JOYSTICK_DEADZONE &&
7           y > -JOYSTICK_DEADZONE && y < JOYSTICK_DEADZONE) {
8           return DIR_NONE;
9       }
10
11      // Dominant axis determines direction
12      if (abs(x) >= abs(y)) {
13          if (x > JOYSTICK_DEADZONE) return DIR_RIGHT;
14          if (x < -JOYSTICK_DEADZONE) return DIR_LEFT;
15      } else {
16          if (y > JOYSTICK_DEADZONE) return DIR_UP;
17          if (y < -JOYSTICK_DEADZONE) return DIR_DOWN;
18      }
19
20      return DIR_NONE;
21  }
```

2.2.4 Button Logic

```
1  bool joystick_buttonPressed(void) {
2      return ((GPIOA->DIN31_0 & JOY_BTN_PIN) == 0);
3  }
```

The joystick button is handled separately as a digital input. The code enables an internal pull-up resistor on the button pin, so the input normally reads high when the button is not pressed. When the joystick button is pressed, it shorts the pin to ground, causing the input to read low. That is why joystick_buttonPressed() checks whether the pin value is 0.

3 Adafruit Screen

3.1 Physical Design Considerations

The screen is a core component of the project - it displays our game and also takes inputs from the user. Shown below is a real-life representation of the screen we use:

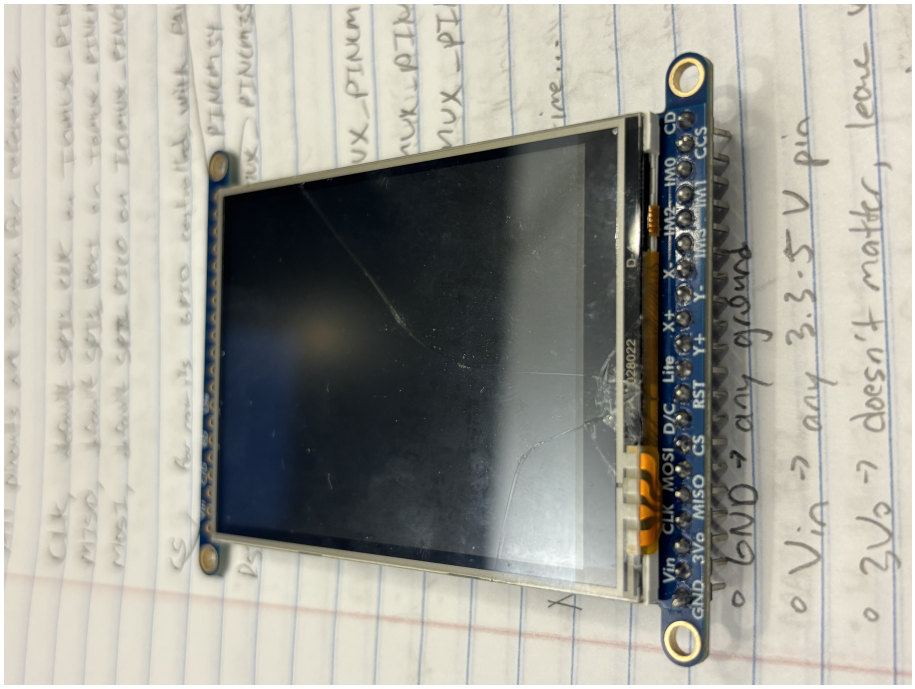

Figure 2: Physical Adafruit LCD Screen

The screen has a few main components:

- **CLK** which controls data transfer to the screen
- **MOSI** which is data input to the screen
- **MISO** which sends data back to the microcontroller

Note that touchscreen is not being used, and the other pins and more screen functionality will be discussed later on in the PCB design section of this report.

3.2 Code Implementation

3.2.1 Framebuffer and Logical Screen Resolution

The display is treated as a 120 x 160 logical screen, where each logical pixel becomes a 2 x 2 block on the real 240 x 320 LCD. This saves memory because each pixel is stored as only one bit: 1 for white and 0 for black.

```
#define BITMAP_COLS (120)
#define BITMAP_ROWS (160)
#define BITMAP_NUM_BITS (BITMAP_COLS * BITMAP_ROWS)
#define BITMAP_NUM_BYTES (BITMAP_NUM_BITS / 8)
#define LCD_BLOCK_SIZE (2)

static uint8_t screen_buffer[BITMAP_NUM_BYTES];
```

3.2.2 Bit Packing

The main drawing buffer is screen_buffer, which stores the current frame that the program wants to display. Functions like Screen_SetPixel() and Screen_GetPixel() convert a (column, row) coordinate into a byte index and bit position inside that packed bitmap. This is why the code uses bit shifting and masking instead of a normal 2D pixel

array. For example, several pixels are stored inside one byte, so setting a pixel means turning one bit on or off. This saves RAM, which is important on a microcontroller, and gives the rest of the game code a simple interface: it can just call Screen_SetPixel(col, row, true) without worrying about SPI or LCD commands.

```
static inline void coord_to_bit(int16_t col, int16_t row,
                                uint16_t *byte_idx,
                                uint8_t *bit_pos) {
    uint16_t idx = (uint16_t)((uint16_t)row * BITMAP_COLS + (uint16_t)col);
    *byte_idx = idx >> 3;
    *bit_pos = (uint8_t)(7 - (idx & 7));
}

void Screen_SetPixel(int16_t col, int16_t row, bool on) {
    if (!in_bounds(col, row)) {
        return;
    }

    uint16_t byte_idx;
    uint8_t bit_pos;
    coord_to_bit(col, row, &byte_idx, &bit_pos);

    uint8_t *buf = active_buffer();

    if (on) {
        buf[byte_idx] |= (uint8_t)(1U << bit_pos);
    } else {
        buf[byte_idx] &= (uint8_t)~(1U << bit_pos);
    }
}
```

3.2.3 Dirty Region Rendering

A major optimization in the screen code is the dirty region rendering. Instead of redrawing the entire display every frame, the code keeps a second buffer called prev_buffer, which stores the last frame that was sent to the LCD. When Screen_Display() is called, it XORs the current buffer with the previous buffer to find which pixels changed. Then it calculates the smallest rectangular bounding box around the changed pixels and only redraws that region. This matters because sending data over SPI is relatively slow, and redrawing the full 240 x 320 LCD every frame would waste a lot of time. For moving objects like a cube or game sprite, only a small part of the screen changes, so this optimization makes the display update much faster.

```
uint8_t diff = (uint8_t)(screen_buffer[i] ^ prev_buffer[i]);

if (diff != 0U) {
    any_dirty = true;

    if (r < min_row) min_row = r;
    if (r > max_row) max_row = r;

    uint16_t left_col =
        (uint16_t)(c_byte * 8U + LEFTMOST_SET[diff]);
    uint16_t right_col =
```

```
        (uint16_t)(c_byte * 8U + RIGHTMOST_SET[diff]);

    if (left_col < min_col) min_col = left_col;
    if (right_col > max_col) max_col = right_col;
}

...

DrawBitmapRegion(screen_buffer, min_col, min_row, max_col, max_row);
```

3.2.4 LCD Initialization

The actual communication with the display happens in drawmethods.c. The InitScreen() function performs the LCD startup sequence: it hard-resets the display, sends a software reset, configures power settings, memory access direction, pixel format, frame rate, gamma settings, exits sleep mode, and finally turns the display on. The code uses the ILI9341-style command flow, where commands like MADCTL, COLMOD, SLPOUT, and DISPON configure how the display behaves. After initialization, the display is ready to accept pixel data through SPI.

```
void InitScreen(void) {
    LCD_HardReset();
    delay_cycles(MS_TO_CYCLES(150));

    LCD_SendCommand(SWRESET, 0x00, 0);
    delay_cycles(MS_TO_CYCLES(150));

    LCD_SendCommand(MADCTL, (uint8_t[]){0x48}, 1);
    LCD_SendCommand(COLMOD, (uint8_t[]){0x55}, 1);
    LCD_SendCommand(FRMCTR1, (uint8_t[]){0x00, 0x18}, 2);

    LCD_SendCommand(SLPOUT, 0x00, 0);
    delay_cycles(MS_TO_CYCLES(120));

    LCD_SendCommand(DISPON, 0x00, 0);
    delay_cycles(MS_TO_CYCLES(20));
}
```

3.2.5 Bitmap to LCD Conversion over SPI

For full-screen drawing, DrawBitmap() sets the LCD address window to the entire 240 x 320 screen, sends the RAMWR command, and then streams pixel data over SPI. Since the internal bitmap is only 120 x 160, each logical pixel is sent as a 2 x 2 block of physical pixels. A 1 bit becomes white RGB565 data, and a 0 bit becomes black RGB565 data.

```
void DrawBitmap(const uint8_t *bitmap) {
    LCD_SendCommand(CASET, (uint8_t[]){0x00, 0x00, 0x00, 0xEF}, 4);
    LCD_SendCommand(PASET, (uint8_t[]){0x00, 0x00, 0x01, 0x3F}, 4);

    SetCS_Low();
    SetDC_Command();
    SPI_SendByte(RAMWR);
    SetDC_Data();
```

```
9
10      for (uint16_t logical_row = 0; logical_row < BITMAP_ROWS; logical_row++) {
11          ...
12          for (uint8_t row_repeat = 0U; row_repeat < LCD_BLOCK_SIZE; row_repeat++) {
13              for (uint16_t logical_col = 0; logical_col < BITMAP_COLS; logical_col++) {
14                  uint8_t hi = row_colors[logical_col][0];
15                  uint8_t lo = row_colors[logical_col][1];
16
17                  SPI_SendByte(hi);
18                  SPI_SendByte(lo);
19                  SPI_SendByte(hi);
20                  SPI_SendByte(lo);
21              }
22          }
23      }
24
25      SetCS_High();
26  }
```

The partial version, DrawBitmapRegion(), does the same thing but only for a specified logical rectangle. It converts the logical region into physical LCD coordinates, sets the LCD column and page address windows with CASET and PASET, and then streams only that smaller region. This is what allows the dirty-region optimization in screen.c to actually reduce the number of SPI bytes sent to the display.

```
1  void DrawBitmapRegion(const uint8_t *bitmap,
2                        uint16_t x0, uint16_t y0,
3                        uint16_t x1, uint16_t y1) {
4      uint16_t phys_x0 = (uint16_t)(x0 * LCD_BLOCK_SIZE);
5      uint16_t phys_x1 = (uint16_t)((x1 + 1) * LCD_BLOCK_SIZE - 1);
6      uint16_t phys_y0 = (uint16_t)(y0 * LCD_BLOCK_SIZE);
7      uint16_t phys_y1 = (uint16_t)((y1 + 1) * LCD_BLOCK_SIZE - 1);
8
9      LCD_SendCommand(CASET, caset_args, 4);
10     LCD_SendCommand(PASET, paset_args, 4);
11
12     SetCS_Low();
13     SetDC_Command();
14     SPI_SendByte(RAMWR);
15     SetDC_Data();
16
17     ...
18     SetCS_High();
19 }
```

3.2.6 Scratch Buffer for Offscreen Rendering

There is also a scratch buffer system for offscreen rendering. When Screen_BeginScratch() is called, drawing operations are redirected from the main screen buffer into scratch_buffer. This lets the program render an object separately, find its bounds, and then copy or shift it into the main framebuffer afterward. In the comments, this is used for workflows like rendering a 3D object at the center first and then shifting it to another location, which helps avoid distortion or makes the drawing pipeline easier to reuse. So overall, the screen code is not just “send pixels

to LCD"; it implements a small graphics engine with a packed framebuffer, optimized partial redraws, and offscreen rendering support.

```c
static uint8_t scratch_buffer[BITMAP_NUM_BYTES];
static bool using_scratch = false;

static inline uint8_t *active_buffer(void) {
    return using_scratch ? scratch_buffer : screen_buffer;
}

void Screen_BeginScratch(void) {
    for (uint16_t i = 0; i < BITMAP_NUM_BYTES; i++) {
        scratch_buffer[i] = 0x00U;
    }
    using_scratch = true;
}

void Screen_EndScratch(void) {
    using_scratch = false;
}
```

4 PCB Design

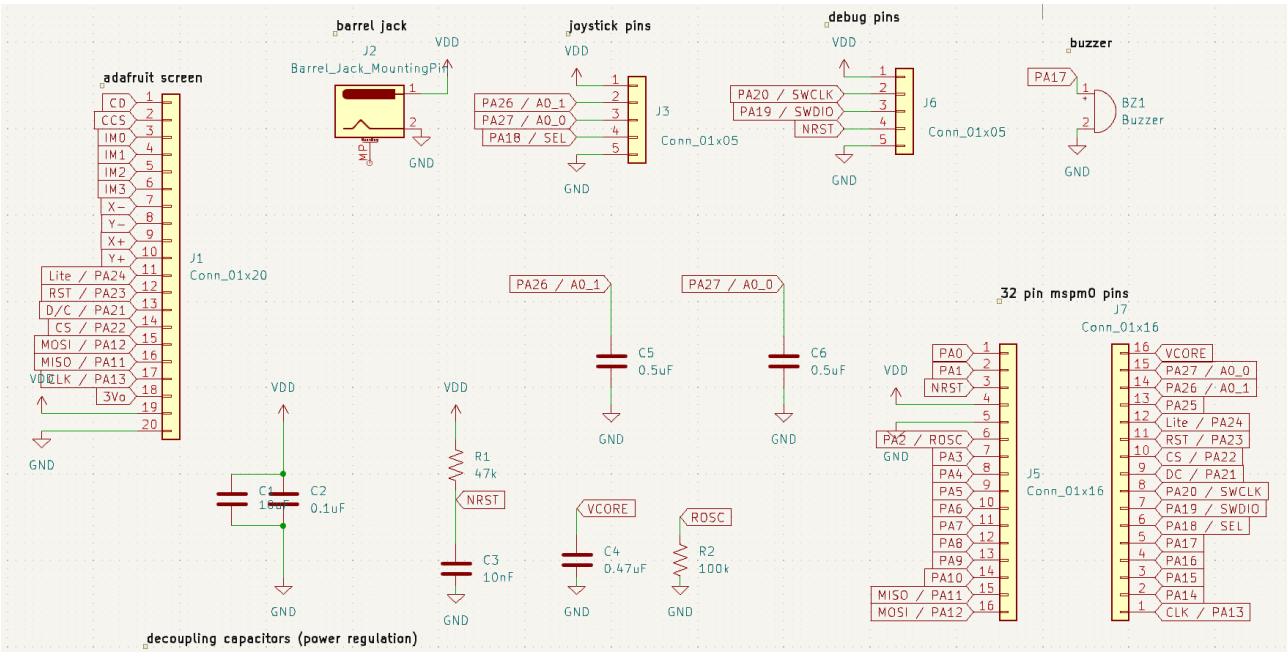

Figure 3: Custom PCB Schematic

4.1 Adafruit LCD Screen

As a brief overview, we describe each connection we use for the Adafruit screen. We knew initially that we will not be using the touchscreen functionaliy, but there needs to be an output display and it needs to take an input as well from the joystick.

- **Vin** takes a 3.3V input from the barrel jack and provides power to the screen

- **GND** is the ground reference.
- **CLK** is the SPI clock and is used to regulate/toggle data transfer to the screen.
- **MOSI** is SPI data input to the display. This is how pixel data and commands are sent, and how our joystick communicates with the screen.
- **MISO** is the data line that lets the peripheral send data back to the microcontroller
- **CS** is chip select, and this lets the screen know when a GPI0 pin is sending input.
- **D/C** is data/command select. Tells the screen whether the SPI byte is a command or display data.
- **RST** resets the display controller.
- **Lite** is the backlight control pin for the screen.

4.2 Barrel Jack

Fairly straightforward - on the PCB we have a barrel jack so that we are able to supply 3.3V of power to various components on the board.

4.3 Joystick

We see that the joystick breakout board consists of 5 pins. Firstly we have the **GND** reference, and we also have the power supply **VDD**. Next, we have **SEL** which is outputting button actions performed by the user. **A0_0** and **A0_1** detect horizontal and vertical motion produced by the user and we specifically chose PA26 and PA27 for their ADC capabilities.

4.4 Debug pins

The debug pins are used to program and debug the **MSPM0** chip after it is soldered onto the custom PCB. Since the board is replacing the TI LaunchPad, the PCB needs its own connection point for an external programmer/debugger. The debug header includes **VDD** and **GND** so the debugger shares the same power reference as the board, **PA20/SWCLK** for the debug clock signal, **PA19/SWDIO** for the debug data signal, and **NRST** so the debugger can reset the microcontroller during programming. These pins are not for the screen or buzzer; they are mainly there so the laptop/programmer can upload code to the MSPM0 and help troubleshoot the board

4.5 PCB - 32 pin QFN package

The 32-pin QFN package is the physical MSPM0 microcontroller chip used on the custom PCB; this chip acts as the main controller for the whole system. Its pins provide the connections for power, ground, reset, debugging, and the peripherals on the board, including the Adafruit screen, buzzer, and other input/output signals.

4.6 Decoupling capacitors

The decoupling capacitors help keep the PCB's power supply stable while the MSPM0 and peripherals are switching on and off. In this schematic, the capacitors connected between **VDD** and **GND**, such as the larger 10μF capacitor and the smaller 0.1 μF capacitor, act like small local energy reserves. When the microcontroller, screen, buzzer, or joystick suddenly needs extra current, the capacitors can briefly supply it and reduce voltage dips or noise on the power rail. The smaller capacitor is especially useful for filtering high-frequency noise close to the chip, while the larger capacitor helps smooth slower changes in the supply voltage. Overall, these capacitors make the board more reliable by preventing unstable power from causing resets, glitches, or incorrect behavior.

4.7 NRST

The **NRST** resistor/capacitor setup is used to make sure the MSPM0 starts up cleanly. The pull-up resistor keeps **NRST** normally high, which allows the microcontroller to run. The capacitor to ground creates a short delay when power is first applied, so the reset pin stays low briefly before rising. This helps prevent the chip from starting while the power rail is still unstable. The debug header can also pull **NRST** low when the programmer needs to reset the chip during flashing or debugging.

4.8 VCORE

The **VCORE** capacitor is needed because **VCORE** is the microcontroller's internal core voltage node. Even though the board is powered from **VDD**, the MSPM0 internally regulates or manages a lower voltage for the CPU core. The capacitor on **VCORE** stabilizes that internal voltage so the processor can run reliably. This pin should not be used as a normal power output or signal pin; it is mainly there so the required capacitor can be connected close to the ch

4.9 ROSC

The **ROSC** resistor is used for the microcontroller's internal oscillator/reference circuit. The MSPM0 uses this external resistor connection to help set or stabilize an internal clock-related function. Since the system clock affects timing, communication, and firmware execution, the resistor value matters and should follow the datasheet/reference design. Like **VCORE**, **ROSC** is not a general-purpose signal connection for peripherals; it is a support pin needed for the chip to operate correctly.

4.10 Horizontal and Vertical Joystick

The joystick capacitors are used to make the analog joystick readings more stable. Since each joystick axis acts like a potentiometer, the **X** and **Y** outputs are analog voltages that change depending on the joystick position. Those voltages go into the MSPM0 ADC pins, such as PA26/A0_1 and PA27/A0_0. Without filtering, the ADC readings can jump around because of electrical noise, small hand vibrations, or switching noise from the rest of the board.

The capacitors from the joystick signal lines to **GND** act as simple low-pass filters. They smooth out fast unwanted voltage changes while still allowing the slower joystick movement signal to pass through. This makes the joystick position easier to read in software because the values are less jittery, so the program does not think the joystick is moving when it is actually near the same position.

5 Conclusion

In conclusion, we have created Cubington, a game where the user controls a Hall effect-based joystick to output direction (horizontal and vertical) and orient a cube in a specific way. For the display, we used an Adafruit LCD Screen (SPI communication protocol) taking inputs from the joystick.